

FINAL IMPLEMENTATION ALGORITHM

INPUT → PROCESS → OUTPUT | Privacy-Aware Ride-Hailing System

A module-by-module implementation document with explicit inputs, algorithm steps, outputs and dependencies. The architecture uses DQN for top-level destination-based pricing and a federated global policy network for aggregate routing response.

Overall flow

- Raw trip data + weather → M1 Data Preprocessing
- M1 → M2 Grid Mapping → M3 Time Slots
- M3 → M4 Demand Aggregation → M5 CNN Demand Prediction
- M2/M3/dispatch history → M6 Estimated Arriving Vehicles
- M7 Federated Routing → $p[g,t]$
- M8 Destination Popularity → $pi[g',t]$
- M9 Base Price → P_base and L_base
- M10 Passenger Sensitivity → private epsilon distribution
- M5 + M6 + M7 + M8 + previous dispatch statistics → M11 DQN Pricing
- M11 + M10 + vehicle availability → M12 Acceptance/Dispatch
- M12 → M16 Reward + next-slot statistics
- M7 + M12 + M13 + M14 → vehicle state evolution
- M15 runs the complete 2-minute simulation loop
- M16 + M17 train DQN; M18 evaluates the final system

Master Module Table

Module	Input	Algorithm / Processing	Output	Consumed By
M1. Data Preprocessing	Raw NYC taxi trip data + required external weather data	1. Load trip records. 2. Parse pickup/drop-off timestamps. 3. Remove impossible/invalid distance or fare values using documented rules. 4. Keep required fields: pickup time, drop-off time, PU/DOLocationID, trip distance, fare. 5. Handle missing values; do not invent passenger sensitivity. 6. Create a unique trip_id. 7. Standardize units and data types. 8. Save cleaned dataset.	Cleaned trip table with standardized timestamps, distance, fare, origin/destination fields; weather-ready join keys	M2, M3, M4, M9, M10
M2. Grid Mapping	Cleaned trips + geographic/grid definition	1. Define fixed spatial grid covering the study region. 2. Map each pickup location to PUGridID. 3. Map each drop-off location to DOGridID. 4. Construct valid grid set G. 5. For each grid, construct current-grid + one-hop neighbor set N_g. 6. Save grid lookup and neighbor lookup.	Trip table with PUGridID and DOGridID; grid dictionary and one-hop neighbor map	M3, M6, M7, M8, M12, M13, M14
M3. Time-Slot Formation	Cleaned/grid-mapped trips + operational horizon	1. Define the operational slot length as 30 minutes. 2. Assign each trip/request to a 30-min slot t. 3. Within each 30-min slot create 15 mini-slots of 2 minutes. 4. Create consistent slot_id and mini_slot_id. 5. Ensure chronological ordering.	Slot-indexed trip data; 30-min slots and 2-min mini-slots	M4, M5, M6, M8, M9, M10, M15
M4. Demand Aggregation	Grid-mapped trips + time slots + weather features if used	1. Count passenger requests by pickup grid and slot. 2. Aggregate demand at the grid-slot level. 3. Create lag/history features required by the CNN. 4. Align weather features with the same time definition if weather is used. 5. Split data chronologically into train/validation/test.	Historical demand tensor/table for each grid and time slot	M5
M5. CNN Demand Prediction	Historical grid demand + temporal/weather features	1. Construct CNN input windows from historical demand. 2. Add the same weather representation used consistently in the research design. 3. Train CNN on training period. 4. Validate on validation period. 5. Freeze the trained predictor for simulation/testing. 6. For each future slot t, output predicted demand $J_hat[g,t]$.	Predicted demand $J_hat[g,t]$ for every required grid and future slot	M11, M15
M6. Estimated Arriving Vehicles	Trip destinations from preceding/current dispatch information + grid mapping	1. Identify successful dispatches whose passenger destination is grid g. 2. Aggregate the corresponding destination arrivals. 3. Use only information available to the PO under the privacy model. 4. Do not reconstruct exact idle-vehicle locations. 5. Store $I[g,t]$ as an aggregate arrival statistic.	$I[g,t]$: estimated number of vehicles arriving into grid g to drop off passengers	M11

Master Module Table

Module	Input	Algorithm / Processing	Output	Consumed By
M7. Federated Routing / Global Policy Network	Grid statistics: mean dispatch price, price deviation, dispatch frequency, charge availability + destination popularity; private vehicle experiences	1. Build grid statistics from the preceding slot. 2. Use dispatch frequency instead of individual driver waiting time in the public routing state. 3. Each idle vehicle receives global policy parameters. 4. Vehicle locally evaluates private utility using its private preferences and current battery state. 5. Generate private repositioning/charging decisions. 6. Train local policy on private experiences. 7. Upload local parameters, not individual experiences. 8. PO aggregates parameters by federated averaging. 9. Deploy aggregated policy to produce mean routing probabilities $p[g,t]$.	Global policy parameters ϕ + routing probability vector $p[g,t]$ over current/neighbor grids	M11, M13, M14, M15
M8. Destination Popularity	Historical/current successful passenger destinations	1. Count destinations over the defined historical/update window. 2. Normalize destination counts into probabilities. 3. Optionally use a moving-average/EWMA update as specified in the final design. 4. Store popularity for every destination grid.	Global destination popularity $\pi[g',t]$	M11
M9. Base-Price Model	Historical trip distance + fare + Time-of-Day $\times$ Weather condition	1. Create Time-of-Day groups. 2. Join weather using a documented timestamp/location rule. 3. Create $condition_id = Time \times Weather$. 4. For each condition c, calculate mean historical trip distance. 5. For each condition c, calculate mean historical fare. 6. Save base values for use in passenger sensitivity and dispatch pricing.	Base-distance $L_{base,c}$ and base-price $P_{base,c}$; condition-level base-price table	M10, M12
M10. Passenger Price Sensitivity	Cleaned/grid-mapped trips + weather + condition-level base distance/price	1. For each trip i in condition c, compute $\Delta L = L_i - L_{base,c}$. 2. Compute $\Delta P = P_i - P_{base,c}$. 3. Retain valid observations with $\Delta L > 0$, $\Delta P > 0$, finite positive epsilon. 4. Calculate $\epsilonpsilon_i = [(L_i - L_{base,c}) / L_{base,c}] / (P_i - P_{base,c})$. 5. Document and apply the chosen outlier rule. 6. Compute mean_epsilon, std_epsilon and sample_count by condition. 7. Inspect empirical distributions before freezing a sampling distribution. 8. Keep individual epsilon private in simulation. 9. Do not provide individual epsilon to the DQN.	Row-level sensitivity table + condition-level mean/std/count + sampling configuration	M12
M11. DQN Destination-Based Pricing	For each origin g and destination g' : $J_hat[g,t]$, previous dispatch statistic, $I[g,t]$, routing vector $p[g,t]$, destination popularity $\pi[g',t]$	1. Construct state $S[g,g',t] = [J_hat, J_disp_prev, I, p, \pi]$. 2. Feed state to the shared DQN/value network. 3. Compute $Q(S,A)$ for the frozen discrete pricing-factor action set. 4. Use epsilon-greedy during training; greedy action during evaluation. 5. Select $\alpha[g,g',t]$. 6. Scale the base fare: $P_disp = \alpha \times P_{base}$. 7. Store transition information for later replay.	Pricing factor $\alpha[g,g',t]$ and Q-values for candidate actions	M12, M16, M17
M12. Passenger Acceptance + Dispatch	Passenger requests; destination; private epsilon; P_{base} ; selected alpha; available idle vehicles	1. For each passenger request, determine origin g, destination g' and condition c. 2. Retrieve/sample private passenger sensitivity $\epsilonpsilon_j$. 3. Compute $P_disp = \alpha[g,g',t] \times P_{base,c}$. 4. Compute passenger maximum acceptable price from the paper's acceptance equation. 5. Accept if $P_disp \leq P_max,j$. 6. Broadcast accepted dispatch order to eligible idle vehicles. 7. Idle vehicles contend using the specified private utility/preferences. 8. Assign the request using the stated dispatch rule. 9. Set $x[j,t]=1$ for successful dispatch. 10. Record dispatch price and mini-slot outcome.	Accepted requests, dispatch assignments, $x[j,t]$, dispatch price, waiting/dispatch statistics	M6, M7, M15, M16

Master Module Table

Module	Input	Algorithm / Processing	Output	Consumed By
M13. Vehicle Repositioning	Global routing statistics + current grid + neighboring grids + private driver preferences	1. At a repositioning decision point, enumerate current grid and one-hop neighbors. 2. Read public grid statistics. 3. For each candidate grid, compute driver utility using private preferences. 4. Include dispatch-price opportunity and dispatch-opportunity components as specified. 5. Choose the grid with highest utility. 6. Move vehicle to the selected grid. 7. Keep individual preference and decision private from the PO.	Vehicle's next grid/repositioning action	M15
M14. EV Charging Navigation	Vehicle battery state + charging availability statistics + candidate grid/station information + private charging preference	1. Check whether battery is below the charging threshold. 2. If charging is not required, continue normal repositioning/dispatch behavior. 3. If charging is required, evaluate feasible charging/navigation options. 4. Use public charging availability/waiting statistics plus private preferences. 5. Select charging navigation decision. 6. Update vehicle location and battery state according to the simulation. 7. Keep individual charging preference private.	Charge/navigation decision and updated vehicle state	M15
M15. 2-Minute Operational Simulation	All frozen modules M1–M14 + current slot state	1. Start a 30-min slot. 2. At slot start, obtain demand prediction, previous dispatch statistics, estimated arrivals, routing probabilities and destination popularity. 3. DQN selects destination-based pricing factors. 4. For each of 15 two-minute mini-slots, generate/process passenger requests. 5. Run passenger acceptance. 6. Run dispatch among eligible idle vehicles. 7. Update vehicle states, trip completion/arrival information, and charging/repositioning states. 8. Update slot-level counters and statistics. 9. End slot and pass aggregated outcomes to the next slot.	Chronological mini-slot events for one 30-min slot	M16, M17, M18
M16. Revenue / Reward Calculation	Successful dispatches $x_{j,t}$ + dispatch prices	1. For each origin-destination pair, sum dispatch prices for successful dispatches. 2. Use $R[g,g',t] = \sum_j P_disp[g,g',t] I(D_j=g') x_{j,t}$. 3. Aggregate pair-level rewards to obtain slot revenue. 4. Store reward with the DQN transition. 5. Accumulate episode revenue for evaluation.	Slot revenue reward $R[g,g',t]$ and total slot/episode revenue	M17, M18
M17. DQN Replay + Training	Stored transitions (state, action, reward, next state) from simulation	1. Store each pricing decision transition in replay memory. 2. Sample mini-batches after sufficient experience is available. 3. Compute target Q-values using the chosen DQN target-network procedure. 4. Minimize temporal-difference loss. 5. Periodically update the target network. 6. Decay exploration epsilon according to the frozen training schedule. 7. Continue across episodes until the stopping criterion is reached. 8. Save the best/final model and training logs.	Updated DQN parameters and trained pricing policy	M18
M18. Evaluation & Ablation	Trained model + test-period data + baseline configurations	1. Run the trained system on unseen/test periods. 2. Measure mean total revenue. 3. Measure mean percentage of successful dispatches. 4. Measure repositioning costs. 5. Measure charging/navigation outcomes where applicable. 6. Evaluate passenger-sensitivity effects. 7. Compare with defined baselines such as MAB if retained. 8. Run ablations by removing/altering selected modules only if specified in the experiment plan. 9. Report results with consistent test population and time period.	Revenue, successful-dispatch rate, repositioning cost, charging-related metrics, routing probabilities, sensitivity analysis and comparison tables/plots	Final results

M1. Data Preprocessing

INPUT: Raw NYC taxi trip data + required external weather data

OUTPUT: Cleaned trip table with standardized timestamps, distance, fare, origin/destination fields; weather-ready join keys

Step	Algorithm
1	Load trip records.
2	Parse pickup/drop-off timestamps.
3	Remove impossible/invalid distance or fare values using documented rules.
4	Keep required fields: pickup time, drop-off time, PU/DOLocationID, trip distance, fare.
5	Handle missing values; do not invent passenger sensitivity.
6	Create a unique trip_id.
7	Standardize units and data types.
8	Save cleaned dataset.

M2. Grid Mapping

INPUT: Cleaned trips + geographic/grid definition

OUTPUT: Trip table with PUGridID and DOGridID; grid dictionary and one-hop neighbor map

Step	Algorithm
1	Define fixed spatial grid covering the study region.
2	Map each pickup location to PUGridID.
3	Map each drop-off location to DOGridID.
4	Construct valid grid set G.
5	For each grid, construct current-grid + one-hop neighbor set N_g.
6	Save grid lookup and neighbor lookup.

M3. Time-Slot Formation

INPUT: Cleaned/grid-mapped trips + operational horizon

OUTPUT: Slot-indexed trip data; 30-min slots and 2-min mini-slots

Step	Algorithm
1	Define the operational slot length as 30 minutes.
2	Assign each trip/request to a 30-min slot t.
3	Within each 30-min slot create 15 mini-slots of 2 minutes.
4	Create consistent slot_id and mini_slot_id.
5	Ensure chronological ordering.

M4. Demand Aggregation

INPUT: Grid-mapped trips + time slots + weather features if used

OUTPUT: Historical demand tensor/table for each grid and time slot

Step	Algorithm
1	Count passenger requests by pickup grid and slot.
2	Aggregate demand at the grid-slot level.
3	Create lag/history features required by the CNN.
4	Align weather features with the same time definition if weather is used.
5	Split data chronologically into train/validation/test.

M5. CNN Demand Prediction

INPUT: Historical grid demand + temporal/weather features

OUTPUT: Predicted demand $J_hat[g,t]$ for every required grid and future slot

Step	Algorithm
1	Construct CNN input windows from historical demand.
2	Add the same weather representation used consistently in the research design.
3	Train CNN on training period.
4	Validate on validation period.
5	Freeze the trained predictor for simulation/testing.
6	For each future slot t, output predicted demand $J_hat[g,t]$.

M6. Estimated Arriving Vehicles

INPUT: Trip destinations from preceding/current dispatch information + grid mapping

OUTPUT: $I[g,t]$: estimated number of vehicles arriving into grid g to drop off passengers

Step	Algorithm
1	Identify successful dispatches whose passenger destination is grid g .
2	Aggregate the corresponding destination arrivals.
3	Use only information available to the PO under the privacy model.
4	Do not reconstruct exact idle-vehicle locations.
5	Store $I[g,t]$ as an aggregate arrival statistic.

M7. Federated Routing / Global Policy Network

INPUT: Grid statistics: mean dispatch price, price deviation, dispatch frequency, charge availability + destination popularity; private vehicle experiences

OUTPUT: Global policy parameters ϕ + routing probability vector $p[g,t]$ over current/neighbor grids

Step	Algorithm
1	Build grid statistics from the preceding slot.
2	Use dispatch frequency instead of individual driver waiting time in the public routing state.
3	Each idle vehicle receives global policy parameters.
4	Vehicle locally evaluates private utility using its private preferences and current battery state.
5	Generate private repositioning/charging decisions.
6	Train local policy on private experiences.
7	Upload local parameters, not individual experiences.
8	PO aggregates parameters by federated averaging.
9	Deploy aggregated policy to produce mean routing probabilities $p[g,t]$.

M8. Destination Popularity

INPUT: Historical/current successful passenger destinations

OUTPUT: Global destination popularity $\pi[g',t]$

Step	Algorithm
1	Count destinations over the defined historical/update window.
2	Normalize destination counts into probabilities.
3	Optionally use a moving-average/EWMA update as specified in the final design.
4	Store popularity for every destination grid.

M9. Base-Price Model

INPUT: Historical trip distance + fare + Time-of-Day × Weather condition

OUTPUT: Base-distance $L_{base,c}$ and base-price $P_{base,c}$; condition-level base-price table

Step	Algorithm
1	Create Time-of-Day groups.
2	Join weather using a documented timestamp/location rule.
3	Create condition_id = Time × Weather.
4	For each condition c, calculate mean historical trip distance.
5	For each condition c, calculate mean historical fare.
6	Save base values for use in passenger sensitivity and dispatch pricing.

M10. Passenger Price Sensitivity

INPUT: Cleaned/grid-mapped trips + weather + condition-level base distance/price

OUTPUT: Row-level sensitivity table + condition-level mean/std/count + sampling configuration

Step	Algorithm
1	For each trip i in condition c , compute $\Delta L = L_i - L_{base,c}$.
2	Compute $\Delta P = P_i - P_{base,c}$.
3	Retain valid observations with $\Delta L > 0$, $\Delta P > 0$, finite positive epsilon.
4	Calculate $\epsilonpsilon_i = [(L_i - L_{base,c}) / L_{base,c}] / (P_i - P_{base,c})$.
5	Document and apply the chosen outlier rule.
6	Compute mean_epsilon, std_epsilon and sample_count by condition.
7	Inspect empirical distributions before freezing a sampling distribution.
8	Keep individual epsilon private in simulation.
9	Do not provide individual epsilon to the DQN.

Frozen sensitivity formula: $\epsilonpsilon_i = [(L_i - L_{base,c}) / L_{base,c}] / (P_i - P_{base,c})$. Base distance and base price are means within each Time × Weather condition.

M11. DQN Destination-Based Pricing

INPUT: For each origin g and destination g' : $J_hat[g,t]$, previous dispatch statistic, $I[g,t]$, routing vector $p[g,t]$, destination popularity $pi[g',t]$

OUTPUT: Pricing factor $alpha[g,g',t]$ and Q-values for candidate actions

Step	Algorithm
1	Construct state $S[g,g',t] = [J_hat, J_disp_prev, I, p, pi]$.
2	Feed state to the shared DQN/value network.
3	Compute $Q(S,A)$ for the frozen discrete pricing-factor action set.
4	Use epsilon-greedy during training; greedy action during evaluation.
5	Select $alpha[g,g',t]$.
6	Scale the base fare: $P_disp = alpha \times P_base$.
7	Store transition information for later replay.

DQN state: $S[g,g',t] = [J_hat[g,t], J_disp[g,t-1], I[g,t], p[g,t], pi[g',t]]$. Action is the selected destination-based pricing factor $alpha[g,g',t]$. $P_disp = alpha \times P_base$. Individual passenger epsilon is not a DQN state input.

M12. Passenger Acceptance + Dispatch

INPUT: Passenger requests; destination; private epsilon; P_base; selected alpha; available idle vehicles

OUTPUT: Accepted requests, dispatch assignments, x[j,t], dispatch price, waiting/dispatch statistics

Step	Algorithm
1	For each passenger request, determine origin g, destination g' and condition c.
2	Retrieve/sample private passenger sensitivity epsilon_j.
3	Compute P_disp = alpha[g,g',t] × P_base,c.
4	Compute passenger maximum acceptable price from the paper's acceptance equation.
5	Accept if P_disp <= P_max,j.
6	Broadcast accepted dispatch order to eligible idle vehicles.
7	Idle vehicles contend using the specified private utility/preferences.
8	Assign the request using the stated dispatch rule.
9	Set x[j,t]=1 for successful dispatch.
10	Record dispatch price and mini-slot outcome.

Acceptance: $P_{disp} \leq P_{base} + (1/\epsilon_j) \times ((L_{g,g'} - L_{base})/L_{base})$. The individual epsilon_j remains private.

M13. Vehicle Repositioning

INPUT: Global routing statistics + current grid + neighboring grids + private driver preferences

OUTPUT: Vehicle's next grid/repositioning action

Step	Algorithm
1	At a repositioning decision point, enumerate current grid and one-hop neighbors.
2	Read public grid statistics.
3	For each candidate grid, compute driver utility using private preferences.
4	Include dispatch-price opportunity and dispatch-opportunity components as specified.
5	Choose the grid with highest utility.
6	Move vehicle to the selected grid.
7	Keep individual preference and decision private from the PO.

M14. EV Charging Navigation

INPUT: Vehicle battery state + charging availability statistics + candidate grid/station information + private charging preference

OUTPUT: Charge/navigation decision and updated vehicle state

Step	Algorithm
1	Check whether battery is below the charging threshold.
2	If charging is not required, continue normal repositioning/dispatch behavior.
3	If charging is required, evaluate feasible charging/navigation options.
4	Use public charging availability/waiting statistics plus private preferences.
5	Select charging navigation decision.
6	Update vehicle location and battery state according to the simulation.
7	Keep individual charging preference private.

M15. 2-Minute Operational Simulation

INPUT: All frozen modules M1–M14 + current slot state

OUTPUT: Chronological mini-slot events for one 30-min slot

Step	Algorithm
1	Start a 30-min slot.
2	At slot start, obtain demand prediction, previous dispatch statistics, estimated arrivals, routing probabilities and destination popularity.
3	DQN selects destination-based pricing factors.
4	For each of 15 two-minute mini-slots, generate/process passenger requests.
5	Run passenger acceptance.
6	Run dispatch among eligible idle vehicles.
7	Update vehicle states, trip completion/arrival information, and charging/repositioning states.
8	Update slot-level counters and statistics.
9	End slot and pass aggregated outcomes to the next slot.

M16. Revenue / Reward Calculation

INPUT: Successful dispatches $x[j,t]$ + dispatch prices

OUTPUT: Slot revenue reward $R[g,g',t]$ and total slot/episode revenue

Step	Algorithm
1	For each origin-destination pair, sum dispatch prices for successful dispatches.
2	Use $R[g,g',t] = \sum_j P_disp[g,g',t] I(D_j=g') x[j,t]$.
3	Aggregate pair-level rewards to obtain slot revenue.
4	Store reward with the DQN transition.
5	Accumulate episode revenue for evaluation.

Reward: $R[g,g',t] = \sum_j P_disp[g,g',t] I(D_j=g') x[j,t]$.

M17. DQN Replay + Training

INPUT: Stored transitions (state, action, reward, next state) from simulation

OUTPUT: Updated DQN parameters and trained pricing policy

Step	Algorithm
1	Store each pricing decision transition in replay memory.
2	Sample mini-batches after sufficient experience is available.
3	Compute target Q-values using the chosen DQN target-network procedure.
4	Minimize temporal-difference loss.
5	Periodically update the target network.
6	Decay exploration epsilon according to the frozen training schedule.
7	Continue across episodes until the stopping criterion is reached.
8	Save the best/final model and training logs.

M18. Evaluation & Ablation

INPUT: Trained model + test-period data + baseline configurations

OUTPUT: Revenue, successful-dispatch rate, repositioning cost, charging-related metrics, routing probabilities, sensitivity analysis and comparison tables/plots

Step	Algorithm
1	Run the trained system on unseen/test periods.
2	Measure mean total revenue.
3	Measure mean percentage of successful dispatches.
4	Measure repositioning costs.
5	Measure charging/navigation outcomes where applicable.
6	Evaluate passenger-sensitivity effects.
7	Compare with defined baselines such as MAB if retained.
8	Run ablations by removing/altering selected modules only if specified in the experiment plan.
9	Report results with consistent test population and time period.

Exact One-Slot Algorithm

1. INPUT: slot t , current vehicle states, historical/current demand information, weather condition, base-price/sensitivity tables, global policy parameters and trained/current DQN parameters.
2. Predict demand $J_hat[g,t]$ for all required grids.
3. Compute previous-slot dispatch statistic $J_disp[g,t-1]$.
4. Compute aggregate estimated arrivals $I[g,t]$.
5. Compute/update public grid statistics and obtain routing probability vector $p[g,t]$.
6. Obtain destination popularity $pi[g',t]$.
7. For every origin-destination pair (g,g') , construct $S[g,g',t]$.
8. Run DQN and select $alpha[g,g',t]$.
9. For mini-slot $m = 1,...,15$: generate/process passenger requests; determine condition; retrieve/sample private epsilon; calculate dispatch price; test acceptance; dispatch accepted requests; update vehicle states.
10. At the end of the slot, calculate $R[g,g',t]$ and total slot revenue.
11. Update dispatch price/frequency/charging statistics.
12. Store DQN transitions in replay memory.
13. Update the federated routing policy according to the FL schedule.
14. Proceed to slot $t+1$.

Coding order

Phase	Modules	Deliverable
Phase 1	M1 → M2 → M3	Data foundation
Phase 2	M4 → M5	Demand prediction
Phase 3	M6 → M8	Aggregate state variables
Phase 4	M9 → M10	Base price and passenger sensitivity
Phase 5	M7	Federated routing policy
Phase 6	M11	DQN pricing
Phase 7	M12 → M14	Dispatch, repositioning and charging
Phase 8	M15 → M16	Integrated simulation and reward
Phase 9	M17	DQN training
Phase 10	M18	Evaluation and comparison

Final Integration Checklist

- M1 cleaned dataset is fixed and reproducible.
- M2 grid IDs and one-hop neighbors are fixed.
- M3 30-minute/2-minute time structure is fixed.
- M5 CNN produces demand predictions for every required grid/slot.
- M6 aggregate arriving-vehicle statistic is available without revealing exact idle-driver locations.
- M7 routing policy produces $p[g,t]$ and is trained through federated parameter aggregation.
- M8 destination popularity is available.
- M9 base distance and base price are frozen by Time $\times$ Weather condition.
- M10 sensitivity distributions are validated and sampling procedure is frozen.
- M11 DQN state/action/reward definitions are frozen before final training.
- M12 passenger acceptance and dispatch logic runs at 2-minute resolution.
- M13/M14 vehicle repositioning and EV charging decisions update vehicle states.
- M15 full slot loop runs without missing inputs.
- M16 reward exactly matches the defined revenue equation.
- M17 replay/training produces a saved DQN.
- M18 test evaluation uses an unseen period and consistent metrics.